

# U-Boot 讲义

## 深入浅出：ARM Linux 引导流程与 U-Boot 架构全景讲义

### 第一章：为什么我们需要引导加载程序（Bootloader）？

在您的台式机或服务器上，开机时有 BIOS 或 UEFI 帮您搞定一切。但在嵌入式 ARM 系统中，情况要艰苦得多。

“引导悖论（Bootstrap Paradox）”：当系统刚通电或复位时，CPU 处于一种“无知”的裸机状态。此时，外部的内存（DRAM，即动态随机存取存储器）由于尚未配置内存控制器，根本无法使用。而庞大的 Linux 内核（通常好几兆字节）必须放在 RAM 中才能运行。Bootloader 的核心使命，就是作为 CPU 和操作系统之间的桥梁，完成底层硬件的初步初始化（如设置系统时钟、配置内存控制器启用 DRAM、初始化串口等），最后将 Linux 内核搬进内存并启动它。

---

### 第二章：多阶段引导序列（Multi-stage Booting）—— 接力赛跑

既然 DRAM 一开始不能用，Bootloader 自己一开始能在哪运行呢？处理器内部通常只有极小容量的 SRAM（静态随机存取存储器，通常从 4KB 到 512KB 不等）。因为容量太小，装不下功能完整的 U-Boot，所以引导过程必须设计成多阶段的“接力赛跑”：

#### 阶段 0：ROM Code（固化在芯片内部的代码）

- 在哪运行：芯片内部的只读存储器（ROM）中，这是出厂就固化好的。
- 做什么：它是通电后运行的第一段代码。它会读取硬件引脚的状态（Boot Mode pins）或内部保险丝（eFUSE），决定从哪里寻找启动程序（是 SD 卡、eMMC、NAND Flash 还是 USB？）。
- 接力给谁：找到外存中的第一阶段引导程序（SPL）后，把它复制到芯片内部微小的 SRAM 中并执行。

## 阶段 1: SPL (Secondary Program Loader, 次级程序加载器)

- 在哪运行: 芯片内部的 SRAM。
- 做什么: SPL 是一个“高度精简版”的 U-Boot。因为 SRAM 空间极小 (比如仅有 48 KiB)，它只包含最核心的驱动。它的终极任务只有一个: 初始化外部大容量的 DDR RAM 控制器，让外接的内存条变得可用。
- 接力给谁: DRAM 初始化完成后，SPL 会去外存中把“完全体”的 U-Boot (U-Boot Proper) 加载到刚刚准备好的外部 DRAM 中，并把接力棒交过去。

## 阶段 2: U-Boot Proper (U-Boot 主体)

- 在哪运行: 外部大容量 DRAM 中。
- 做什么: 此时系统已经拥有了海量的内存，U-Boot 可以大展拳脚了。它提供了命令行交互界面 (CLI)、网络协议栈 (方便通过 TFTP 下载内核)、各种文件系统支持 (FAT, ext4 等)。
- 接力给谁: 它负责将 Linux 内核镜像、设备树 (DTB) 和初始内存文件系统 (initramfs) 加载到 RAM 中，最终把系统控制权移交给 Linux 操作系统。

---

## 第三章: U-Boot 的独门绝技 (架构机制)

在复杂的引导接力中，U-Boot 运用了两个非常精妙的架构机制来保证平滑过渡：

1. 全局数据指针 (Global Data, GD) 在 SPL 阶段或刚进入 DRAM 时，C 语言运行所需的 BSS 段 (存放未初始化全局变量的内存区) 还无法使用。为了在这些阶段之间安全地存储系统配置和状态 (如检测到的内存大小、设备树指针等)，U-Boot 定义了一个叫 gd 的结构体。妙招: U-Boot 将这个结构体的指针强制保存在 CPU 的一个特定寄存器中 (ARM 32 位是 r9 寄存器，ARM 64 位是 x18 寄存器)，确保不管内存状态如何，核心配置数据始终触手可及。
  2. 代码自重定位 (Self-Relocation) 当 U-Boot 在 RAM 中完全运行起来后，它会做一件奇妙的事: 计算当前可用的最高内存地址，然后把自己整个代码复制 (重定位) 到 RAM 的最顶端区域。目的: Linux 内核通常希望被放置在 RAM 的低端地址区域。U-Boot 把自己搬到最上面，就能把下方巨大的连续内存空间彻底腾出来，留给庞大的 Linux 内核使用。这要求 U-Boot 必须被编译为“位置无关可执行文件” (PIE)，并在搬家后修复所有的绝对地址指针。
-

## 第四章：内核如何被包装？（内核镜像格式）

当 U-Boot 准备加载内核时，面对的内核文件有不同的格式。了解它们有助于明白 U-Boot 是如何工作的：

- **Image**: 这是 Linux 内核编译出来的最原始、未压缩的二进制文件，体积最大。
  - **zImage**: 为了节省宝贵的存储空间，内核通常会被压缩。zImage 不仅是被压缩过的内核，它的头部还自带了一段\*\*“自解压代码 (Self-extracting stub)”\*\*。它被设计为位置无关代码 (PIC)，可以加载到内存的任何地方执行。在现代 U-Boot 中，我们使用 `bootz` 命令来直接启动它。
  - **uImage**: 这是 U-Boot 早期的“私有封装格式”。本质上是利用 `mkimage` 工具，在原始镜像或 zImage 的头上加了一个 64 字节的信息头（包含 OS 类型、入口地址、CRC 校验等）。必须使用 `bootm` 命令启动。但如今 U-Boot 的维护者指出，在自带解压功能的 zImage 外层再包一层 uImage 是多余且愚蠢的（“pretty stupid”），现代开发更推荐直接使用 zImage。
  - **FIT (Flattened Image Tree)**: 现代最强大的复合镜像格式。它可以把内核、设备树 (DTB) 和 `initramfs` 全都打包成一个文件，不仅方便管理，还支持密码学签名，用于实现安全验证启动 (Secure Boot)。
- 

## 第五章：终极交接仪式 —— 引导 Linux 内核

万事俱备，U-Boot 即将“功成身退”。在跳转到 Linux 内核的第一条指令前，它必须严格遵守 ARM 架构定义的引导协议 (Boot Protocol)：

### 1. 硬件状态必须“干净”

- **中断关闭**: 所有的硬件中断 (IRQs 和 FIQs) 必须被死死屏蔽，防止在内核初始化时发生不可预期的打断。
- **缓存与 MMU 关闭**: 内存管理单元 (MMU) 必须关闭，数据缓存 (Data Cache) 必须关闭。CPU 必须看到最真实的物理内存。
- **处理器模式**: CPU 必须处于特权管理模式 (SVC 模式)，或者支持虚拟化的芯片需要处于 HYP (Hypervisor) 模式。

## 2. 内存布局必须“安全”

U-Boot 要把各种数据放在 RAM 的安全位置，防止它们相互覆盖：

- Kernel (zImage)：建议放在距离 RAM 起始地址较近的低端内存中（比如 +32MiB 的位置），避免解压时需要再次搬移自己。
- Device Tree (.dtb)：设备树二进制文件必须加载到 64 位对齐的地址，通常建议放在 RAM 起始地址 +128MiB 的边界之上。
- initramfs (可选)：如果使用了初始内存文件系统，建议将它放置在紧挨着 DTB 上方的安全位置。同时必须确保它落在内核的“低端内存映射 (low-memory mapping)”范围内，并且不会被内核解压器覆盖。

## 3. CPU 寄存器传参（核心机密）

U-Boot 是通过直接操作 CPU 的寄存器来向内核传递启动参数的。在跳转的瞬间，寄存器必须是这样的状态：

- r0 寄存器：必须强制设置为 0。
- r1 寄存器：包含了机器类型 ID (Machine Type Number)。在早期的 ATAGs 时代这很重要，而在现代使用设备树的系统中，这个值通常填为全 1 (~0) 即可。
- r2 寄存器：至关重要的一步！它必须包含设备树 (DTB) 在物理内存中的起始地址。内核在启动时，会通过 r2 指针去读取这棵“树”，从而知道板子上到底挂载了哪些硬件设备、需要加载什么驱动。

---

## 总结

这就是一个 ARM 嵌入式设备唤醒的完整旅程：

1. ROM Code 寻找启动介质并唤醒 SPL。
2. SPL 顶着极小内存的压力，点亮并初始化 DRAM。
3. U-Boot Proper 进驻 DRAM，重定位自己腾出空间，从外存/网络中把 zImage (内核)、DTB (设备树) 和 initramfs 拉进内存的最佳位置。
4. U-Boot 关闭中断和缓存，将 DTB 的地址塞进 r2 寄存器。
5. U-Boot 执行绝对跳转指令，Linux 内核苏醒，接管了这片被精心整理过的数字江山。

### 1. 硬链接 hard link

给同一个文件内容再起一个名字。

比如：ln a.txt b.txt

这表示给 a.txt 创建一个硬链接 b.txt。

此时：a.txt 和 b.txt 指向同一块真实文件数据，它们地位是平等的，没有谁是原文件、谁是快捷方式。

如果你修改 a.txt：echo hello > a.txt

再看 b.txt，内容也会变，因为它们本质上是同一个文件。

删除其中一个：rm a.txt

b.txt 仍然可以正常使用。只有当所有硬链接都删除后，文件数据才真正被释放。

硬链接 = 同一个文件的多个文件名

### 2. 软链接 symbolic link (亦称符号链接)

软链接也叫符号链接，可以理解成：类似 Windows 里的快捷方式。

创建命令：ln -s a.txt b.txt

这表示创建一个软链接 b.txt，它指向 a.txt。

此时：b.txt 只是保存了 a.txt 这个路径

如果访问 b.txt，系统会自动跳到 a.txt。

但是如果删除原文件：rm a.txt

那么 b.txt 就失效了，因为它指向的目标不存在了。

这种失效的软链接也叫：断开的软链接 / 悬空链接

### 3. 主要区别

| 对比          | 硬链接              | 软链接           |
|-------------|------------------|---------------|
| 本质          | 同一个文件的另一个名字      | 指向目标路径的快捷方式   |
| 删除原文件后      | 另一个硬链接仍可用        | 软链接失效         |
| 是否有独立 inode | 没有，和原文件 inode 相同 | 有自己的 inode    |
| 能否跨文件系统     | 一般不能             | 可以            |
| 能否链接目录      | 一般不能             | 可以            |
| 创建命令        | ln 原文件 链接名       | ln -s 原文件 链接名 |

最简单记法：

硬链接：文件的另一个名字，删掉原名也不影响使用

软链接：文件的快捷方式，原文件没了它就失效

比如：

ln test.txt hard.txt

ln -s test.txt soft.txt

这里：

hard.txt 是 test.txt 的硬链接

soft.txt 是 test.txt 的软链接

### inode (索引节点) 是什么？

Linux 文件系统中用来记录“文件本身信息”的编号和数据结构。

文件名只是给人看的名字，而真正管理文件的是 inode。

比如你有一个文件：a.txt

系统内部大概是这样看的：

文件名 a.txt → inode 号 → 文件的真实信息和数据位置

inode 里面会记录很多信息，例如：文件大小、文件权限、文件所有者、文件所属组、创建

/修改/访问时间、文件数据存放在哪些磁盘块、硬链接数量

但是注意：inode 里一般不存文件名。

文件名通常存放在目录项里，目录项负责把“文件名”和“inode 号”对应起来。

比如：

a.txt → inode 12345

b.txt → inode 12345

如果 a.txt 和 b.txt 是硬链接，它们就指向同一个 inode，所以它们本质上是同一个文件。

可以用命令查看 inode 号：ls -li

如果两个文件 inode 号一样，说明它们指向同一个文件数据。

一句话记：

inode 就是文件在 Linux 文件系统里的“身份证”，文件名只是指向这个身份证的名字。